SQL Practice Queries

Oracle HR Schema

A Complete Guide to the SELECT Statement
For Beginners - After Your First 2 Lessons

Level: A1 (Beginner)
Database: Oracle
Schema: HR (Human Resources)

1. Introduction

This book is for you if you just started learning SQL. You finished your first two lessons. Now you want to practice. This book has many practice questions. All questions use the Oracle HR schema. The HR schema has tables about employees, departments, jobs, and locations. It is a very common schema for SQL practice all over the world.

In this book, we will learn all parts of the SELECT statement. SELECT is the most important command in SQL. You use it to read data from a database. Every data analyst uses SELECT every day. If you know SELECT well, you can do your job well.

Each section has a short explanation first. Then we show the SQL syntax. After that, we give you practice questions. Try to write the SQL yourself first. Then look at the answer. This is the best way to learn. Do not just read the answers. Write the code yourself!

1.1 How to Use This Book

First, read the explanation in each section. Look at the syntax examples. Then try to answer the practice questions by yourself. You can use Oracle Live SQL or any Oracle database. After you try, check your answer with our solution. If your answer is different, that is OK. There are many correct ways to write the same query.

1.2 What You Need to Know

Before you start, you should know these things: What is a table, what is a column, what is a row, and the basic SELECT * FROM table syntax. If you know these, you are ready!

2. The Oracle HR Schema

The HR schema is a sample database from Oracle. It has 7 tables. These tables store information about a company. The company has regions, countries, locations, departments, jobs, and employees. Let us look at each table.

2.1 REGIONS Table

This table has region information.

Column	Type	Description
`REGION_ID`	NUMBER	Region number (Primary Key)
`REGION_NAME`	VARCHAR2(25)	Name of the region

Table 1: REGIONS Table

2.2 COUNTRIES Table

This table has country information. Each country belongs to one region.

Column	Type	Description
COUNTRY_ID	CHAR(2)	Country code (Primary Key)
COUNTRY_NAME	VARCHAR2(40)	Name of the country
REGION_ID	NUMBER	Region number (Foreign Key)

Table 2: COUNTRIES Table

2.3 LOCATIONS Table

This table has office location information. Each location is in one country.

Column	Type	Description
LOCATION_ID	NUMBER	Location number (Primary Key)
STREET_ADDRESS	VARCHAR2(40)	Street address
POSTAL_CODE	VARCHAR2(12)	Postal code
CITY	VARCHAR2(30)	City name
STATE_PROVINCE	VARCHAR2(25)	State or province
COUNTRY_ID	CHAR(2)	Country code (Foreign Key)

Table 3: LOCATIONS Table

2.4 DEPARTMENTS Table

This table has department information. Each department has a location and a manager.

Column	Type	Description
DEPARTMENT_ID	NUMBER	Department number (Primary Key)
DEPARTMENT_NAME	VARCHAR2(30)	Name of the department
MANAGER_ID	NUMBER	Manager employee ID
LOCATION_ID	NUMBER	Location number (Foreign Key)

Table 4: DEPARTMENTS Table

2.5 JOBS Table

This table has job title information. It stores the job id, title, minimum and maximum salary.

Column	Type	Description
JOB_ID	VARCHAR2(10)	Job code (Primary Key)

JOB_TITLE	VARCHAR2(35)	Job title
MIN_SALARY	NUMBER	Minimum salary
MAX_SALARY	NUMBER	Maximum salary

Table 5: JOBS Table

2.6 EMPLOYEES Table

This is the most important table. It has all employee information. Each employee has a job, a department, and a manager. Some columns can be NULL (empty). For example, COMMISSION_PCT is NULL for most employees because they do not get commission.

Column	Type	Description
EMPLOYEE_ID	NUMBER	Employee number (Primary Key)
FIRST_NAME	VARCHAR2(20)	First name
LAST_NAME	VARCHAR2(25)	Last name
EMAIL	VARCHAR2(25)	Email address
PHONE_NUMBER	VARCHAR2(20)	Phone number
HIRE_DATE	DATE	Date when employee started
JOB_ID	VARCHAR2(10)	Job code (Foreign Key)
SALARY	NUMBER	Monthly salary
COMMISSION_PCT	NUMBER	Commission percent
MANAGER_ID	NUMBER	Manager employee ID
DEPARTMENT_ID	NUMBER	Department number (Foreign Key)

Table 6: EMPLOYEES Table

2.7 JOB_HISTORY Table

This table shows the past jobs of employees. It records when an employee changed jobs.

Column	Type	Description
EMPLOYEE_ID	NUMBER	Employee number
START_DATE	DATE	Start date of this job
END_DATE	DATE	End date of this job
JOB_ID	VARCHAR2(10)	Job code
DEPARTMENT_ID	NUMBER	Department number

Table 7: JOB_HISTORY Table

3. Basic SELECT Statement

The SELECT statement is the most basic and most used SQL command. It reads data from a table. Think of it like this: you have a big table with many columns. SELECT lets you choose which columns you want to see. You can also choose all columns with the star symbol (*).

3.1 SELECT All Columns

If you want to see all columns in a table, use SELECT *. The star (*) means "all columns". This is the easiest way to look at a table. But be careful with big tables. If a table has millions of rows, SELECT * can be very slow.

Syntax

```
SELECT *
FROM table_name;
```

Practice

Q1: Show all data from the REGIONS table.

Q2: Show all data from the JOBS table.

Q3: Show all data from the DEPARTMENTS table.

Answers

```
-- Q1: Show all regions
SELECT *
FROM regions;

-- Q2: Show all jobs
SELECT *
FROM jobs;

-- Q3: Show all departments
SELECT *
FROM departments;
```

3.2 SELECT Specific Columns

Most of the time, you do not need all columns. You only need some columns. In this case, write the column names after SELECT. Separate each column name with a comma. This is better than SELECT * because it is faster and clearer.

Syntax

```
SELECT column1, column2, column3
FROM table_name;
```

Practice

Q4: Show only FIRST_NAME and LAST_NAME from EMPLOYEES.

Q5: Show JOB_TITLE, MIN_SALARY, and MAX_SALARY from JOBS.

Q6: Show DEPARTMENT_NAME and LOCATION_ID from DEPARTMENTS.

Answers

```
-- Q4: Show first and last name
SELECT first_name, last_name
FROM employees;

-- Q5: Show job title and salary range
SELECT job_title, min_salary, max_salary
FROM jobs;

-- Q6: Show department name and location
SELECT department_name, location_id
FROM departments;
```

4. DISTINCT - Remove Duplicates

Sometimes a column has the same value many times. For example, many employees work in the same department. If you run SELECT department_id FROM employees, you will see the same department numbers many times. If you want to see each value only one time, use DISTINCT. DISTINCT removes duplicate rows from the result.

4.1 Basic DISTINCT

Syntax

```
SELECT DISTINCT column_name
FROM table_name;
```

Practice

Q7: Show all different DEPARTMENT_IDs from EMPLOYEES (no duplicates).

Q8: Show all different JOB_IDs from EMPLOYEES.

Q9: Show all different COUNTRY_IDs from LOCATIONS.

Answers

```
-- Q7: Unique department IDs
SELECT DISTINCT department_id
FROM employees;

-- Q8: Unique job IDs
SELECT DISTINCT job_id
FROM employees;
```

```
-- Q9: Unique country IDs
SELECT DISTINCT country_id
FROM locations;
```

4.2 DISTINCT with Multiple Columns

You can use DISTINCT with more than one column. In this case, DISTINCT looks at the combination of all columns. If the combination is unique, the row is kept. If two rows have the same values in all selected columns, one of them is removed.

Syntax

```
SELECT DISTINCT column1, column2
FROM table_name;
```

Practice

Q10: Show all different DEPARTMENT_ID and JOB_ID combinations from EMPLOYEES.

Q11: Show all different CITY and COUNTRY_ID combinations from LOCATIONS.

Answers

```
-- Q10: Unique department and job combinations
SELECT DISTINCT department_id, job_id
FROM employees;

-- Q11: Unique city and country combinations
SELECT DISTINCT city, country_id
FROM locations;
```

5. Column Aliases

An alias is a new name for a column. You use the keyword AS to give a column a different name in the result. This is very useful because the original column names are often short or not clear. For example, "MIN_SALARY" is not very clear for a report. You can rename it to "Minimum Salary". You can also use aliases for calculated columns. If you do not use AS, it still works, but using AS makes your code easier to read.

5.1 Column Alias with AS

Syntax

```
SELECT column_name AS 'New Name'
FROM table_name;
```

Practice

Q12: Show FIRST_NAME and LAST_NAME as "First" and "Last" from EMPLOYEES.

Q13: Show JOB_TITLE as "Job" and MIN_SALARY as "Min Pay" from JOBS.

Answers

```
-- Q12: Column aliases for names
SELECT first_name AS 'First',
last_name AS 'Last'
FROM employees;

-- Q13: Column aliases for jobs
SELECT job_title AS 'Job',
min_salary AS 'Min Pay'
FROM jobs;
```

5.2 Table Aliases

You can also give a table a short name. This is called a table alias. You write the short name after the table name. Table aliases do not use AS in most cases, but you can use AS if you want. Table aliases are very useful when you write long queries. They make your SQL shorter and easier to read.

Syntax

```
SELECT t.column_name
FROM table_name t;
```

Practice

Q14: Use "e" as alias for EMPLOYEES. Show e.FIRST_NAME and e.LAST_NAME.

Q15: Use "d" as alias for DEPARTMENTS. Show d.DEPARTMENT_NAME and d.LOCATION_ID.

Answers

```
-- Q14: Table alias for employees
SELECT e.first_name, e.last_name
FROM employees e;

-- Q15: Table alias for departments
SELECT d.department_name, d.location_id
FROM departments d;
```

6. WHERE Clause - Filter Data

The WHERE clause is one of the most important parts of SQL. It lets you filter rows. Without WHERE, SELECT returns all rows. With WHERE, you get only the rows that match your condition. For example, you can find employees with salary greater than 10000. Or employees who work in department 10. The WHERE clause comes after FROM. Always write it after FROM.

6.1 WHERE with Comparison Operators

Comparison operators compare a value with another value. Here are the main operators: = (equal), != or <> (not equal), > (greater than), < (less than), >= (greater or equal), <= (less or equal). These operators work with numbers, strings, and dates.

Operator	Meaning	Example
`=`	Equal to	`salary = 10000`
`!= or <>`	Not equal to	`department_id != 10`
`>`	Greater than	`salary > 5000`
`<`	Less than	`salary < 15000`
`>=`	Greater or equal	`salary >= 8000`
`<=`	Less or equal	`salary <= 12000`

Table 8: Comparison Operators

Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Practice

Q16: Show all employees where SALARY is greater than 10000.

Q17: Show all employees where DEPARTMENT_ID is 10.

Q18: Show all jobs where MIN_SALARY is less than 5000.

Q19: Show all employees where JOB_ID is not 'IT_PROG'.

Q20: Show all employees where SALARY is greater than or equal to 12000.

Answers

```
-- Q16: Salary greater than 10000
SELECT first_name, last_name, salary
FROM employees
WHERE salary > 10000;

-- Q17: Department 10 employees
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id = 10;

-- Q18: Jobs with min salary under 5000
SELECT job_title, min_salary
FROM jobs
WHERE min_salary < 5000;
```

```
-- Q19: Not IT_PROG jobs
SELECT first_name, last_name, job_id
FROM employees
WHERE job_id <> 'IT_PROG';

-- Q20: Salary 12000 or more
SELECT first_name, last_name, salary
FROM employees
WHERE salary >= 12000;
```

7. AND, OR, NOT - Multiple Conditions

Sometimes you need more than one condition. You can combine conditions with AND, OR, and NOT. AND means both conditions must be true. OR means at least one condition must be true. NOT means the opposite of the condition. You can use these operators together in the WHERE clause.

7.1 AND Operator

The AND operator needs ALL conditions to be true. If one condition is false, the row is not returned. For example: WHERE salary > 5000 AND department_id = 10. This means the employee must have salary more than 5000 AND must be in department 10. Both conditions must be true.

Practice

Q21: Show employees where SALARY > 5000 AND DEPARTMENT_ID = 80.

Q22: Show employees where JOB_ID = 'SA_REP' AND SALARY >= 10000.

Answers

```
-- Q21: High salary in department 80
SELECT first_name, last_name, salary, department_id
FROM employees
WHERE salary > 5000
AND department_id = 80;

-- Q22: Sales reps with high salary
SELECT first_name, last_name, salary
FROM employees
WHERE job_id = 'SA_REP'
AND salary >= 10000;
```

7.2 OR Operator

The OR operator needs only ONE condition to be true. If at least one condition is true, the row is returned. For example: WHERE department_id = 10 OR department_id = 20. This shows employees from department 10 or 20 or both. The employee only needs to be in

one of these departments.

Practice

Q23: Show employees where DEPARTMENT_ID is 10 or 20.

Q24: Show employees where JOB_ID is 'IT_PROG' or 'SA_REP'.

Answers

```
-- Q23: Department 10 or 20
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id = 10
OR department_id = 20;

-- Q24: IT staff or Sales reps
SELECT first_name, last_name, job_id
FROM employees
WHERE job_id = 'IT_PROG'
OR job_id = 'SA_REP';
```

7.3 NOT Operator

The NOT operator reverses a condition. It makes true false and false true. You can use it with any condition. For example: WHERE NOT department_id = 10. This shows all employees who are NOT in department 10. NOT is very useful for excluding data.

Practice

Q25: Show employees who are NOT in department 10.

Q26: Show employees whose SALARY is NOT between 5000 and 10000.

Answers

```
-- Q25: Not department 10
SELECT first_name, last_name, department_id
FROM employees
WHERE NOT department_id = 10;

-- Q26: Salary outside 5000-10000
SELECT first_name, last_name, salary
FROM employees
WHERE NOT salary BETWEEN 5000 AND 10000;
```

7.4 Combining AND, OR, NOT

You can use AND, OR, and NOT together. But be careful with the order. AND is checked before OR. Use parentheses () to control the order. Always use parentheses when you mix AND and OR. This makes your code clear and avoids mistakes.

Practice

Q27: Show employees where (DEPARTMENT_ID = 10 OR 20) AND SALARY > 5000.

Q28: Show employees where DEPARTMENT_ID = 80 AND (SALARY > 10000 OR COMMISSION_PCT > 0.2).

Answers

```
-- Q27: Dept 10 or 20 with high salary
SELECT first_name, last_name, department_id, salary
FROM employees
WHERE (department_id = 10 OR department_id = 20)
AND salary > 5000;

-- Q28: Dept 80, high salary or high commission
SELECT first_name, last_name, salary, commission_pct
FROM employees
WHERE department_id = 80
AND (salary > 10000 OR commission_pct > 0.2);
```

8. IN Operator

The IN operator is a shortcut for many OR conditions. Instead of writing column = 10 OR column = 20 OR column = 30, you can write column IN (10, 20, 30). The IN operator checks if a value is in a list of values. It is shorter, cleaner, and easier to read than using many OR operators.

8.1 IN with Numbers

Syntax

```
SELECT column1, column2
FROM table_name
WHERE column IN (value1, value2, value3);
```

Practice

Q29: Show employees where DEPARTMENT_ID is 10, 20, or 30.

Q30: Show employees where SALARY is 4800, 6000, or 12008.

Answers

```
-- Q29: Three departments
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id IN (10, 20, 30);

-- Q30: Three specific salaries
SELECT first_name, last_name, salary
```

```
FROM employees
WHERE salary IN (4800, 6000, 12008);
```

8.2 IN with Strings

You can also use IN with text values. Put each value in single quotes. For example: job_id IN ('IT_PROG', 'SA_REP', 'FI_ACCOUNT'). This finds employees with any of these three job IDs.

Practice

Q31: Show employees where JOB_ID is 'IT_PROG', 'SA_REP', or 'FI_ACCOUNT'.

Q32: Show departments where LOCATION_ID is 1700, 2400, or 2500.

Answers

```
-- Q31: Three job types
SELECT first_name, last_name, job_id
FROM employees
WHERE job_id IN ('IT_PROG', 'SA_REP', 'FI_ACCOUNT');

-- Q32: Three locations
SELECT department_name, location_id
FROM departments
WHERE location_id IN (1700, 2400, 2500);
```

8.3 NOT IN

You can use NOT IN to exclude values. This shows rows where the value is NOT in the list. It is the opposite of IN.

Practice

Q33: Show employees where DEPARTMENT_ID is NOT 10, 20, or 30.

Q34: Show employees where JOB_ID is NOT 'IT_PROG' and NOT 'SA_REP'.

Answers

```
-- Q33: Not in three departments
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id NOT IN (10, 20, 30);

-- Q34: Not IT or Sales
SELECT first_name, last_name, job_id
FROM employees
WHERE job_id NOT IN ('IT_PROG', 'SA_REP');
```

9. BETWEEN Operator

The BETWEEN operator selects values within a range. It includes both the start and end values. BETWEEN is a shortcut for: column >= value1 AND column <= value2. You can use it with numbers, dates, and strings. It is cleaner and shorter than using AND with two comparisons.

9.1 BETWEEN with Numbers

Syntax

```
SELECT column1, column2
FROM table_name
WHERE column BETWEEN value1 AND value2;
```

Practice

Q35: Show employees where SALARY is between 5000 and 10000.

Q36: Show jobs where MIN_SALARY is between 4000 and 8000.

Answers

```
-- Q35: Salary between 5000 and 10000
SELECT first_name, last_name, salary
FROM employees
WHERE salary BETWEEN 5000 AND 10000;

-- Q36: Min salary between 4000 and 8000
SELECT job_title, min_salary
FROM jobs
WHERE min_salary BETWEEN 4000 AND 8000;
```

9.2 NOT BETWEEN

NOT BETWEEN gives you the opposite result. It shows values outside the range. This means values less than the start value OR greater than the end value.

Practice

Q37: Show employees where SALARY is NOT between 5000 and 10000.

Q38: Show jobs where MAX_SALARY is NOT between 10000 and 20000.

Answers

```
-- Q37: Salary outside 5000-10000
SELECT first_name, last_name, salary
FROM employees
WHERE salary NOT BETWEEN 5000 AND 10000;
```

```
-- Q38: Max salary outside range
SELECT job_title, max_salary
FROM jobs
WHERE max_salary NOT BETWEEN 10000 AND 20000;
```

10. LIKE Operator - Pattern Matching

The LIKE operator searches for a pattern in a string. It is very useful when you do not know the exact value. LIKE uses two special symbols called wildcards: % (percent) means zero or more characters. _ (underscore) means exactly one character. You can use LIKE to find names that start with "S", or emails that end with "@oracle.com", and many more patterns.

10.1 Wildcard Symbols

Symbol	Meaning	Example	Matches
`%`	Any characters (0 or more)	`'S%'`	Steven, Sarah, S
_	Exactly one character	`'_mith'`	Smith
`%or%`	Contains "or"	`'%or%'`	Neena, Egor

Table 9: LIKE Wildcards

10.2 LIKE Practice

Pattern: Starts With

Q39: Show employees whose FIRST_NAME starts with 'S'.

Q40: Show employees whose LAST_NAME starts with 'K'.

Q41: Show departments whose name starts with 'Sa'.

Pattern: Contains

Q42: Show employees whose FIRST_NAME contains 'an'.

Q43: Show employees whose EMAIL contains 'on'.

Pattern: Ends With

Q44: Show employees whose LAST_NAME ends with 'son'.

Q45: Show employees whose EMAIL ends with 'com'.

Pattern: Exact Length

Q46: Show employees whose FIRST_NAME has exactly 5 characters (use four underscores).

Q47: Show departments whose DEPARTMENT_NAME has exactly 4 characters.

Pattern: NOT LIKE

Q48: Show employees whose FIRST_NAME does NOT start with 'S'.

Q49: Show employees whose EMAIL does NOT contain 'on'.

Answers

```
-- Q39: Name starts with S
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE 'S%';

-- Q40: Last name starts with K
SELECT first_name, last_name
FROM employees
WHERE last_name LIKE 'K%';

-- Q41: Department starts with Sa
SELECT department_name
FROM departments
WHERE department_name LIKE 'Sa%';

-- Q42: Name contains 'an'
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE '%an%';

-- Q43: Email contains 'on'
SELECT first_name, email
FROM employees
WHERE email LIKE '%on%';

-- Q44: Last name ends with 'son'
SELECT first_name, last_name
FROM employees
WHERE last_name LIKE '%son';

-- Q45: Email ends with 'com'
SELECT first_name, email
FROM employees
WHERE email LIKE '%com';

-- Q46: First name = 5 characters
SELECT first_name
FROM employees
WHERE first_name LIKE '____';

-- Q47: Department name = 4 characters
SELECT department_name
FROM departments
WHERE department_name LIKE '____';
```

```
-- Q48: Name NOT starting with S
SELECT first_name, last_name
FROM employees
WHERE first_name NOT LIKE 'S%';

-- Q49: Email NOT containing 'on'
SELECT first_name, email
FROM employees
WHERE email NOT LIKE '%on%';
```

11. IS NULL and IS NOT NULL

In SQL, NULL means "no value" or "empty". It is not zero. It is not an empty string. NULL is a special value that means "unknown". You cannot use = NULL to check for NULL values. You must use IS NULL instead. To check for values that are NOT null, use IS NOT NULL. In the HR schema, COMMISSION_PCT is NULL for many employees because they do not earn commission. MANAGER_ID in DEPARTMENTS can also be NULL for some departments.

> **Tip:** Never write WHERE column = NULL. This does NOT work! Always use IS NULL.

11.1 IS NULL

Syntax

```
SELECT column1, column2
FROM table_name
WHERE column IS NULL;
```

Practice

Q50: Show employees who do NOT have a COMMISSION_PCT (it is NULL).

Q51: Show employees who do NOT have a DEPARTMENT_ID.

Q52: Show departments that do NOT have a MANAGER_ID.

Answers

```
-- Q50: No commission
SELECT first_name, last_name, commission_pct
FROM employees
WHERE commission_pct IS NULL;

-- Q51: No department
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id IS NULL;

-- Q52: No manager
SELECT department_name, manager_id
```

```
FROM departments
WHERE manager_id IS NULL;
```

11.2 IS NOT NULL

Practice

Q53: Show employees who HAVE a COMMISSION_PCT (it is not NULL).

Q54: Show departments that HAVE a MANAGER_ID.

Answers

```
-- Q53: Have commission
SELECT first_name, last_name, commission_pct
FROM employees
WHERE commission_pct IS NOT NULL;

-- Q54: Have manager
SELECT department_name, manager_id
FROM departments
WHERE manager_id IS NOT NULL;
```

12. ORDER BY - Sort Results

The ORDER BY clause sorts the result. You can sort by one column or many columns. You can sort in ascending order (A to Z, 1 to 10) or descending order (Z to A, 10 to 1). Ascending is the default. If you do not specify, SQL sorts in ascending order automatically. ORDER BY is always the LAST clause in a SELECT statement. It comes after WHERE.

12.1 ORDER BY ASC (Ascending)

ASC means ascending order. For numbers: small to big. For text: A to Z. For dates: old to new. ASC is the default, so you do not need to write it. But it is good practice to write it for clarity.

Syntax

```
SELECT column1, column2
FROM table_name
ORDER BY column1 ASC;
```

Practice

Q55: Show all employees ordered by FIRST_NAME (A to Z).

Q56: Show all employees ordered by SALARY (low to high).

Q57: Show all employees ordered by HIRE_DATE (oldest to newest).

Answers

```
-- Q55: By first name A-Z
SELECT first_name, last_name
FROM employees
ORDER BY first_name ASC;

-- Q56: By salary low to high
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary ASC;

-- Q57: By hire date oldest first
SELECT first_name, last_name, hire_date
FROM employees
ORDER BY hire_date ASC;
```

12.2 ORDER BY DESC (Descending)

DESC means descending order. For numbers: big to small. For text: Z to A. For dates: new to old. You must write DESC if you want descending order. Without it, SQL always uses ascending.

Practice

Q58: Show all employees ordered by SALARY (high to low).

Q59: Show all employees ordered by HIRE_DATE (newest to oldest).

Q60: Show jobs ordered by MAX_SALARY (high to low).

Answers

```
-- Q58: By salary high to low
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC;

-- Q59: By hire date newest first
SELECT first_name, last_name, hire_date
FROM employees
ORDER BY hire_date DESC;

-- Q60: Jobs by max salary
SELECT job_title, max_salary
FROM jobs
ORDER BY max_salary DESC;
```

12.3 ORDER BY Multiple Columns

You can sort by more than one column. SQL sorts by the first column. If two rows have the same value in the first column, it sorts those rows by the second column. For example:

ORDER BY department_id, salary DESC. This sorts by department first. Inside each department, it sorts by salary from high to low.

Practice

Q61: Show employees sorted by DEPARTMENT_ID, then by SALARY (high to low).

Q62: Show employees sorted by JOB_ID, then by FIRST_NAME (A to Z).

Q63: Show employees sorted by DEPARTMENT_ID (ascending), then HIRE_DATE (descending).

Answers

```
-- Q61: By dept, then salary
SELECT first_name, last_name, department_id, salary
FROM employees
ORDER BY department_id ASC, salary DESC;

-- Q62: By job, then name
SELECT first_name, last_name, job_id
FROM employees
ORDER BY job_id, first_name;

-- Q63: By dept asc, then hire date desc
SELECT first_name, last_name, department_id, hire_date
FROM employees
ORDER BY department_id ASC, hire_date DESC;
```

13. Arithmetic in SELECT

You can do math in SQL. You can add (+), subtract (-), multiply (*), and divide (/) numbers. You can also use parentheses for calculations. Arithmetic is very useful for salary calculations. For example, you can calculate annual salary (salary * 12) or salary with bonus (salary * 1.1). When you do arithmetic, always use a column alias to give the result a good name.

13.1 Basic Arithmetic

Syntax

```
SELECT column1, column2 * 12 AS alias_name
FROM table_name;
```

Practice

Q64: Show FIRST_NAME, SALARY, and annual salary (SALARY * 12).

Q65: Show FIRST_NAME, SALARY, and SALARY with 10% bonus (SALARY * 1.1).

Q66: Show FIRST_NAME, SALARY, and salary difference from 15000 (15000 - SALARY).

Answers

```
-- Q64: Annual salary
SELECT first_name, salary, salary * 12 AS annual_salary
FROM employees;

-- Q65: Salary with 10% bonus
SELECT first_name, salary, salary * 1.1 AS with_bonus
FROM employees;

-- Q66: Salary diff from 15000
SELECT first_name, salary, 15000 - salary AS diff_from_15000
FROM employees;
```

13.2 Arithmetic with NULL

If you do arithmetic with NULL, the result is always NULL. For example: salary + commission_pct. If commission_pct is NULL, the result is NULL. This is a common problem. Later you will learn about the NVL function to handle this. For now, just remember: any math with NULL gives NULL.

> **Tip:** salary + NULL = NULL. salary * NULL = NULL. Any math with NULL gives NULL.

13.3 Operator Precedence

SQL follows standard math rules. Multiplication and division are done first. Addition and subtraction are done second. Use parentheses to change the order. For example: salary * 12 + 1000 is not the same as salary * (12 + 1000). Always use parentheses when you are not sure about the order.

Practice

Q67: Show FIRST_NAME and SALARY * 12 + 5000 (annual salary plus 5000).

Q68: Show FIRST_NAME and SALARY * (12 + 1) as salary for 13 months.

Answers

```
-- Q67: Annual salary plus 5000
SELECT first_name, salary * 12 + 5000 AS total
FROM employees;

-- Q68: Salary for 13 months
SELECT first_name, salary * (12 + 1) AS total_13
FROM employees;
```

14. String Concatenation

Concatenation means joining two or more strings together. In Oracle SQL, you can use the || operator (double pipe) to join strings. You can join column values with text. For

example, you can join FIRST_NAME and LAST_NAME to make a full name. You can also add spaces and other text.

14.1 The || Operator

Syntax

```
SELECT column1 || ' ' || column2 AS full_name
FROM table_name;
```

Practice

Q69: Show FIRST_NAME and LAST_NAME together as "Full Name".

Q70: Show FIRST_NAME, LAST_NAME, and EMAIL joined as: "Name: Steven King (SKING)".

Answers

```
-- Q69: Full name
SELECT first_name || ' ' || last_name AS full_name
FROM employees;

-- Q70: Name with email
SELECT first_name || ' ' || last_name ||
' (' || email || ')' AS employee_info
FROM employees;
```

15. The DUAL Table

DUAL is a special table in Oracle. It has only one row and one column. You use it when you do not need to read from any real table. You can use DUAL to test expressions, do calculations, or check the current date. DUAL is unique to Oracle. Other databases do not have it.

Practice

Q71: Show the current date using SYSDATE from DUAL.

Q72: Calculate 100 * 50 using DUAL.

Q73: Show the text 'Hello SQL' using DUAL.

Answers

```
-- Q71: Current date
SELECT SYSDATE AS today
FROM dual;

-- Q72: Simple math
SELECT 100 * 50 AS result
FROM dual;
```

```
-- Q73: Simple text
SELECT 'Hello SQL' AS greeting
FROM dual;
```

16. Review - Mixed Practice

This section has questions that use many SELECT features together. These questions are harder. Try to solve them by yourself first. Then check the answers. Good luck!

Practice

Q74: Show FIRST_NAME, LAST_NAME, SALARY, and annual salary (SALARY * 12) for employees in department 90. Order by SALARY from high to low.

Q75: Show all different JOB_IDs from employees who earn more than 10000.

Q76: Show FIRST_NAME, SALARY, and a column called "Level" that shows: SALARY >= 10000 as "High", others as "Normal". (Hint: use CASE).

Q77: Show FIRST_NAME and EMAIL for employees whose FIRST_NAME starts with "S" or "A", and SALARY is between 5000 and 15000.

Q78: Show all departments that do NOT have a MANAGER_ID, ordered by DEPARTMENT_NAME.

Q79: Show FIRST_NAME, LAST_NAME, and SALARY * 12 + commission for employees who HAVE a COMMISSION_PCT. Order by total (salary*12 + commission) from high to low.

Q80: Show the number of employees in each DEPARTMENT_ID. (Hint: use COUNT, which you will learn in the next lessons).

Answers

```
-- Q74: Department 90 with annual salary
SELECT first_name, last_name, salary,
salary * 12 AS annual_salary
FROM employees
WHERE department_id = 90
ORDER BY salary DESC;

-- Q75: Different high-paying jobs
SELECT DISTINCT job_id
FROM employees
WHERE salary > 10000;

-- Q76: Salary level (with CASE)
SELECT first_name, salary,
CASE
WHEN salary >= 10000 THEN 'High'
ELSE 'Normal'
END AS level
```

```
FROM employees;

-- Q77: S or A names, salary range
SELECT first_name, email, salary
FROM employees
WHERE (first_name LIKE 'S%' OR first_name LIKE 'A%')
AND salary BETWEEN 5000 AND 15000;

-- Q78: Departments without manager
SELECT department_name
FROM departments
WHERE manager_id IS NULL
ORDER BY department_name;

-- Q79: Employees with commission, total pay
SELECT first_name, last_name,
salary * 12 +
salary * commission_pct AS total
FROM employees
WHERE commission_pct IS NOT NULL
ORDER BY total DESC;

-- Q80: Employees per department (COUNT)
SELECT department_id, COUNT(*) AS emp_count
FROM employees
GROUP BY department_id
ORDER BY department_id;
```

17. Summary

In this book, you learned all the important parts of the SELECT statement. Let us review what you learned. You can use SELECT * to see all columns, or list specific columns. DISTINCT removes duplicate values. Aliases give columns and tables new names. WHERE filters rows by conditions. AND, OR, NOT combine multiple conditions. IN checks a value against a list. BETWEEN checks a range. LIKE searches for patterns with % and _. IS NULL and IS NOT NULL check for empty values. ORDER BY sorts the results. Arithmetic lets you do math in SQL. || joins strings together. DUAL is a special Oracle table.

This is a lot! But do not worry. Practice every day. Start with simple queries. Then try harder ones. The more you practice, the better you get. SQL is like a language. The more you speak it, the better you become. Good luck on your SQL journey!

Topic	Keyword	What It Does
All Columns	`SELECT *`	Shows all columns
No Duplicates	`DISTINCT`	Removes duplicate rows
New Name	`AS`	Creates column or table alias

Filter	`WHERE`	Filters rows by condition
Combine	`AND, OR, NOT`	Combines multiple conditions
In List	`IN`	Checks value in a list
Range	`BETWEEN`	Checks value in a range
Pattern	`LIKE`	Searches with wildcards
Empty Check	`IS NULL`	Checks for NULL values
Sort	`ORDER BY`	Sorts the result
Math	`+ - * /`	Does arithmetic
Join Text	`||`	Joins strings together

Table 10: SELECT Statement Summary

You are now ready for the next level! In your next lessons, you will learn about functions, GROUP BY, HAVING, and JOINs. These are powerful tools that will make you even better at SQL. Keep practicing and keep learning!